Technical Report

Power Grid Model

Security Assessment

Prepared for:
OSTIF



SHIELDER
WEB SECURITY

1. Document Details

Classification	**Public** - CC BY-SA 4.0
Last review	October 13, 2026
Author	Mattia Ricciardi

1.1. Version

Identifier	Date	Author	Note
v1.0	July 07, 2026	Mattia Ricciardi	First version
v1.1	July 07, 2026	Pietro Tirenna	Peer review
v1.2	October 13, 2026	Abdel Adim Oisfi	Public release

1.2. Contacts Information

Company	Name	Position	Contact
Shielder	Abdel Adim `Smaury` Oisfi	CEO	abdeladim.oisfi@shielder.it
Shielder	Mattia Ricciardi	Security Researcher	mattia.ricciardi@shielder.it
Shielder	Pietro Tirenna	Security Researcher	pietro.tirenna@shielder.it
OSTIF	Derek Zimmer	Executive Director	derek@ostif.org
OSTIF	Amir Montazery	Managing Director	amir@ostif.org
OSTIF	Helen Woeste	Communications and Community Manager	helen@ostif.org
OSTIF	Tom Welter	Project Manager	tom@ostif.org
Alliander	Peter Salemink	Product Owner	peter.salemink@alliander.com
Alliander	Santiago Figueroa Manrique	Software Engineer	santiago.figueroa.manrique@alliander.com

Public - CC BY-SA 4.0 License

1.3. About OSTIF

The **Open Source Technology Improvement Fund (OSTIF)** is dedicated to resourcing and managing security engagements for open source software projects through partnerships with corporate, government, and non-profit donors. We bridge the gap between resources and security outcomes, while supporting and championing the open source community whose efforts underpin our digital landscape.

Over the past ten years, OSTIF has been responsible for the discovery of over 800 vulnerabilities, (121 of those being Critical/High), over 13,000 hours of security work, and millions of dollars raised for open source security. Maximizing output and security outcomes while minimizing labor and cost for projects and funders has resulted in partnerships with multi-billion dollar companies, top open source foundations, government organizations, and respected individuals in the space. Most importantly, we've helped over 120 projects and counting improve their security posture.

Our directive is to support and enrich the open source community through providing public-facing security audits, educational resources, meetups, tooling, and advice. OSTIF's experience positions us to be able to share knowledge of auditing with maintainers, developers, foundations, and the community to further secure our infrastructure in a sustainable manner.

We are a small team working out of Chicago, Illinois. Our website is ostif.org. You can follow us on social media to keep up to date on audits, conferences, meetups, and opportunities with OSTIF, or feel free to reach out directly at contactus@ostif.org or our Github.

Derek Zimmer, Executive Director
Amir Montazery, Managing Director
Helen Woeste, Communications and Community Manager
Tom Welter, Project Manager



Public - CC BY-SA 4.0 License

2. Summary

1. Document Details 2
1.1. Version 2
1.2. Contacts Information 2
1.3. About OSTIF 3
2. Summary 4
3. Executive Summary 5
3.1. Overview 5
3.2. Context and Scope 5
3.3. Methodology 6
3.4. Audit Summary 6
3.5. Recommendations 7
3.6. Long Term Improvements 7
3.7. Results Summary 8
3.8. Findings Severity Classification 9
3.9. Remediation Status Classification 10
4. Fuzzing 11
5. Findings Details 12
5.1. Arbitrary Method Invocation via YAML Mapping 12
5.2. Unbounded JSON Nesting Leads to Denial of Service (DoS) 14
5.3. Out-Of-Bounds Memory Access in Span Calculation Leads to Denial of Service (DoS) 16
6. Hardenings 18
6.1. pgm-build-dependencies Fetched via Mutable Latest Release 18
7. Attachments 19
7.1. finding1/rce_poc.py 19
7.2. finding1/payload.yaml 19
7.3. finding2/json_dos_poc.py 19
7.4. finding3/oob_poc.c 19

Public - CC BY-SA 4.0 License

3. Executive Summary

The document aims to highlight the findings identified during the “Security Assessment” against the "Power Grid Model" product described in section “3.2 Context and Scope”.

For each detected findings, the following information are provided:

- **Severity**: findings score ("3.8 Findings Severity Classification").
- **Affected resources**: vulnerable components.
- **Status**: remediation status ("3.9 Remediation Status Classification").
- **Description**: type and context of the detected finding.
- **Impact**: loss of confidentiality, data integrity and/or availability in case of a successful exploitation and conditions necessary for a successful attack.
- **Proof of Concept**: evidence and/or reproduction steps.
- **Suggested remediation**: configurations or actions needed to mitigate the finding.
- **References**: useful external resources.

3.1. Overview

In April 2026, **Shielder** was hired by the **Open Source Technology Improvement Fund (OSTIF)** to perform a Preliminary Security Review and Threat Model Assessment of Power Grid Model, a C and Python library for steady-state distribution power system analysis.

Power Grid Model (PGM) consists of:

- A core C++ library which supports various types of calculations.
- A C API shim that exposes the core C++ object instantiations and method calls.
- A Python package, built on top of the C API, which exposes user-friendly methods to perform calculations and leverages NumPy structured arrays for its data structures.

Although the functionalities of PGM are all implemented inside the main `power-grid-model` project, the auxiliary `power-grid-model-io` library was considered in scope, given its uses and the potential expansion of the attack surface.

A team of 2 (two) Shielder engineers worked on this project for a total of 20 (twenty) person-days of audit effort.

3.2. Context and Scope

Power Grid Model is an open-source library developed under the LF Energy umbrella and maintained by Alliander. It is used for steady-state distribution power system analysis — including Power Flow, State Estimation, and Short Circuit calculations — and has seen adoption across the energy sector as a high-performance computational backend.

Public - CC BY-SA 4.0 License

The aim of this security assessment was to evaluate the security posture of the Power Grid Model project, with particular attention to attack surfaces reachable from untrusted input, focusing on the deserialization pipeline, the C API boundary, and the file ingestion layer.

In particular, the main goals were to:

- Perform a threat modeling assessment to identify high-risk entry points and their associated attack scenarios.
- Perform a manual review of the C++ core library, the C API bridge, and the Python I/O package, mainly focusing on memory safety, integer arithmetic, and unsafe deserialization patterns.
- Review the dependency surface and CI/GitHub Workflow configuration for supply-chain exposure.

The scope of this audit covers the `power_grid_model` repository at commit `78b616befca3f7ecf2f9c28696beb2fe93a69eb0` and the `power_grid_model_io` repository at commit `e2aba462ecb8a049bb835c8afe86761282a54613`, both released on April 20, 2026. It is important to note that Security Assessments are time-boxed activities performed at a specific point in time; thus, they are unable to guarantee that a software is or will be free of bugs.

3.3. Methodology

The source code audit was carried out by combining various testing techniques and approaches.

Manual and tool-driven techniques were used to analyze the source code. The audit was assisted by SAST tools like and Semgrep with publicly available C/C++ queries and rules.

In parallel with the source code audit, a preliminary fuzzing campaign was established using a basic OSSFuzz-compatible, coverage-guided fuzzer. The fuzzer was then enhanced with a custom mutator to reduce the generation of invalid inputs, ensuring more test cases reached the core functionalities.

Finally, Shielder performed a review of the release process for misconfigurations that could lead to supply-chain attacks. This included, for instance, reviewing the configured GitHub actions and workflows for typical attack scenarios.

3.4. Audit Summary

Power Grid Model is implemented following secure coding principles in the C++ core and in the C API boundaries between the Python interface and the core functionalities. It is however affected by some security issues in the parsing of YAML files and in the Github Workflows integrity.

The Shielder team was able to identify 1 (one) high, 1 (one) medium and 1 (one) low findings.

Public - CC BY-SA 4.0 License

3.5. Recommendations

The following list outlines recommendations for the project's maintainers to harden the security posture of the project:

Implement SBOM Generation

Power Grid Model currently publishes no Software Bill of Materials (SBOM) alongside its release artifacts, meaning downstream integrators have no record of which exact versions of the dependencies are bundled in a given wheel.

It is recommended to generate a CycloneDX or SPDX SBOM as part of every release workflow and publish it as a GitHub Release asset and a PyPI attestation.

Implement `narrow_cast` in Release Mode

The project defines a `narrow_cast<T>()` utility in common/typing.hpp (line 18) that guards integer narrowing conversions with `assert(std::in_range<T>(value))`. This assertion is however compiled out in every release build, since PyPI wheels are compiled with `NDEBUG` defined, which means `narrow_cast` will just resolve into a plain `static_cast` at every call site.

It is recommended to replace the assert with a throwing check, keeping the `if constexpr(std::same_as<T, U>)` branch so that same-type conversions remain a no-op.

Enable UBSan in the Unix Sanitizer Clang Preset

The `unix-sanitizer` configuration preset in CMakePresets.json sets `SANITIZER_FLAGS` to `-fsanitize=address` only. UBSan catches runtime errors like signed integer overflow, misaligned pointer dereferences and other forms of undefined behavior.

It is therefore recommended to include `-fsanitize=address,undefined` in the preset and add `-fno-sanitize-recover=undefined` so that any detected undefined behavior aborts the process rather than logging and continuing.

3.6. Long Term Improvements

Due to fast-evolving nature of the Security field and the time-constrained nature of Security Audits, there still is room for long-term improvements to the overall security of the project's ecosystem.

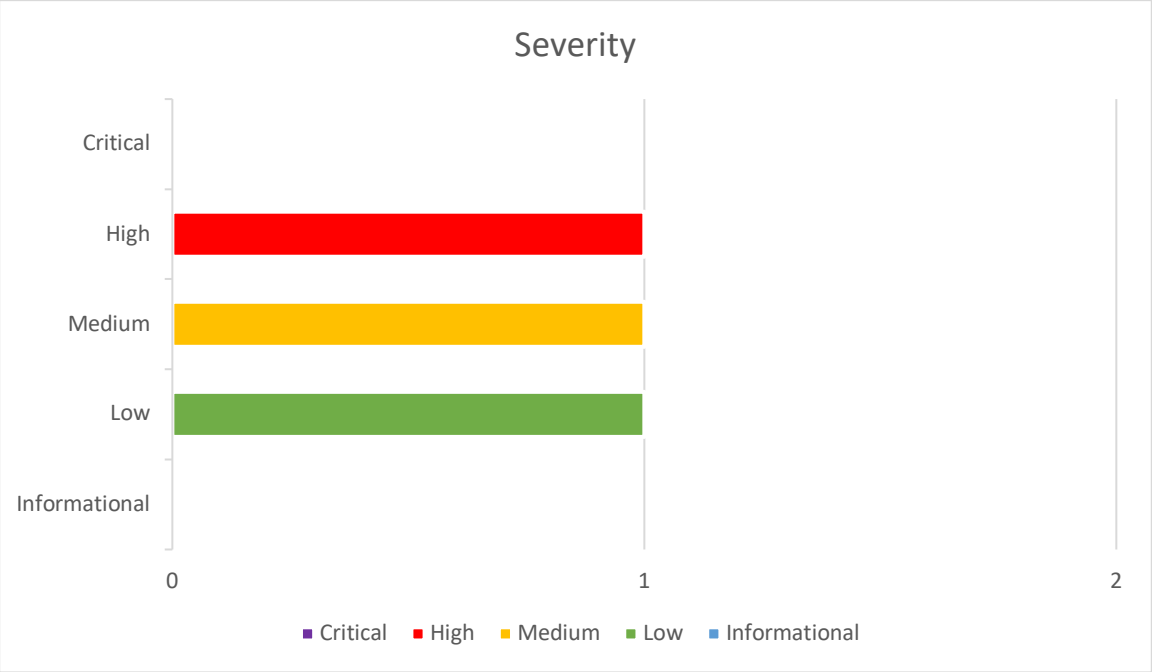
Perform an In-Depth Code Audit

When implementing complex mathematical calculations in memory-unsafe languages like C or C++, it's not uncommon to introduce bugs with a serious security impact that may be exploited by attackers in the wild. It is thus recommended to perform a more in-depth audit of the codebase.

Public - CC BY-SA 4.0 License

3.7. Results Summary

The following chart shows the number of findings found per severity:



ID	Finding	Severity	Status
1	Arbitrary Method Invocation via YAML Mapping	HIGH	Closed
2	Unbounded JSON Nesting Leads to Denial of Service (DoS)	MEDIUM	Closed
3	Out-Of-Bounds Memory Access in Span Calculation Leads to Denial of Service (DoS)	LOW	Closed

Public - CC BY-SA 4.0 License

3.8. Findings Severity Classification

Severity	Description
CRITICAL	Vulnerability that allows to compromise the whole application, host and/or infrastructure. In some cases, it allows access, in read and/or write, to highly sensitive data, totally impacting the resources in terms of confidentiality, integrity and availability. Usually, such vulnerabilities can be exploited without the need of valid credentials, without considerable difficulty and with the possibility of automated, highly reliable, and remotely triggerable attacks. Vulnerabilities marked with this severity must be resolved quickly, especially in production environment.
HIGH	Vulnerability that significantly affects the confidentiality, integrity, and availability of confidential and sensitive data. However, the prerequisites for the attack affect its likelihood of success, such as the presence of controls or mitigations and the need of a certain set of privileges.
MEDIUM	Vulnerability that allows to obtain only a limited or less sensitive set of data, partially compromising confidentiality. In some cases, it may affect the integrity and availability of the information, but with a lower level of severity. In addition, the chances of success of such vulnerability may depend on external factors and/or conditions outside the attacker's control.
LOW	Vulnerability resulting in a limited loss of confidentiality, integrity, and availability of data. In some cases, it depends on conditions not aligned to a real scenario or requires that the attacker has access to credentials with a high level of privileges. In addition, a low severity vulnerability may provide useful information to successfully exploit a higher impact vulnerability.
INFORMATIONAL	Problems that do not directly impact confidentiality, integrity, and availability. Usually, these problems indicate the absence of security mechanisms or the improper configuration of them. Mitigation of this type of problem increases the general level of security of the system and allows in some cases to prevent potential new vulnerabilities and/or limit the impact of existing ones.

Public - CC BY-SA 4.0 License

3.9. Remediation Status Classification

Status	Description
Open	Vulnerability not mitigated or insufficient mitigation.
Not reproducible	Vulnerability not reproducible due to environment changes or to mitigation of other vulnerabilities required in the reproduction steps.
Closed	Vulnerability mitigated. The security patch applied is reasonably robust.

Public - CC BY-SA 4.0 License

4. Fuzzing

During the assessment, a fuzzing campaign was conducted. Since the project was not on-boarded on OSS-Fuzz yet, the Shielder team developed a harness using the de facto standard LibFuzzer `LLVMFuzzerTestOneInput` interface, exercising the full parsing and calculation flow end to end.

After discussing it with the Power Grid Model maintainers, however, the team refactored it into several smaller harnesses, as the maintainers asked, so that each one is easier for them to maintain over time. Each harness covers one stage of the flow: parsing the document, writing it into buffers, building the model, and running the calculations. This means a change to one part of the library only affects the harness for that part, and each stage can be fuzzed at its own speed and depth. Each harness takes the fuzzing input as a single JSON document.

The following harnesses were developed:

- `fuzz_deserialize_json`: covers `PGM_create_deserializer_from_binary_buffer` and the `PGM_dataset_info_*` APIs, which turn JSON data into a dataset and expose its metadata.
- `fuzz_parse_to_buffer`: covers `PGM_deserializer_parse_to_buffer`, which writes the parsed data into row-based and columnar buffers, including batch datasets
- `fuzz_model_create`: covers `PGM_create_model`, `PGM_copy_model` and `PGM_get_indexer`, which build the in-memory model.
- `fuzz_power_flow`, `fuzz_state_estimation`, `fuzz_short_circuit`: cover `PGM_calculate` with every calculation method, symmetry and option supported by each calculation type.
- `fuzz_model_batch_update`: covers batch calculations and `PGM_update_model`, which apply an update document to an existing model.

The harnesses were provided to the maintainers together with a seed corpus, a dictionary and the configuration needed to on-board the project on OSS-Fuzz, so that fuzzing can continue after the assessment.

The harnesses were proposed to the project's repository in PR #1617 and are currently under review.

Public - CC BY-SA 4.0 License

5. Findings Details

Analysis results are discussed in this section.

5.1. Arbitrary Method Invocation via YAML Mapping

Severity	HIGH
Affected Resources	power-grid-model-io/src/power_grid_model_io/utils/modules.py:12-27 power-grid-model-io/src/power_grid_model_io/converters/tabular_converter.py:230 power-grid-model-io /src/power_grid_model_io/converters/tabular_converter.py:849
Status	Closed

September 03, 2026 Update

The vulnerability was reported via the GitHub Security section of the repository in the linked advisory and assigned CVE-2026-84457.
The vulnerability was fixed by implementing a new version of Power-Grid-Model-IO which adds an allowlist for functions in tabular converter configuration. The fixes are all implemented in commit 370b100 and released in version 2.0.

Description

The `get_function()` method from `modules.py` returns a function pointer (or, in Python terminology, a function handle) taken from a dotted path inside a provided YAML mapping file:

```
def get_function(fn_name: str) -> Callable:
    """
    Get a function pointer by name
    """
    parts = fn_name.split(".")
    function_name = parts.pop()
    module_path = ".".join(parts) if parts else "builtins"
    try:
        module = import_module(module_path)
    except ModuleNotFoundError as ex:
        raise AttributeError(f"Module '{module_path}' does not exist (tried to
resolve function '{fn_name}')!") from ex
    try:
        fn_ptr = getattr(module, function_name)
    except AttributeError as ex:
        raise AttributeError(f"Function '{function_name}' does not exist in
module '{module_path}'!") from ex
    return fn_ptr
```

This method is then called inside `tabular_converter.py`, specifically by the `_parse_table_filters()` and `_parse_function()` methods. In particular, the Tabular

Public - CC BY-SA 4.0 License

Converter will end up calling `fn(**dict(zip(key_words, row)))`. A maliciously crafted YAML mapping could be used to define a "function" `col_def` whose key is `os.popen` and whose value maps a keyword argument "cmd" to a data column. This will effectively call `os.popen(cmd=<input>)` when the YAML mapping is used to load input data.

Although the repository's `SECURITY.md` states that a white-list is used to prevent execution of arbitrary functions, no such implementation has been found to be used by the Tabular Converter. Therefore, an attacker who can supply or modify a YAML mapping will be able to achieve import-and-call of any importable symbol in the host process. If they can additionally load arbitrary input data on said mapping, they could also pass arguments to the imported symbol.

Impact

An attacker that can supply YAML mapping files and/or input data to a Tabular Converter could achieve code execution inside the host process.

Attack Complexity

The attacker needs to be able to supply arbitrary YAML mappings or to inject content inside those loaded by a user application. Moreover, to call the method with arguments, the attacker needs to be able to load arbitrary input data into the malicious mapping. The exploitability of this vulnerability depends on the implementation of the user application.

Related Issues

N/A

Proof of Concept

1. Download the attached files `rce_poc.py` and `payload.yaml` in the same directory.
2. From the directory with the files downloaded at step 1, run the PoC using `python3 rce_poc.py`
3. Check the `/tmp` directory which should contain a `pgm_rce` file with the output of the `id` command, proving command execution was successfully achieved

Suggested Remediations

Replace the current filtering mechanism with a secure, non-Turing-complete expression language like CEL (Common Expression Language) rather than deserializing complex objects.
Alternatively, implement a strict white-list that restricts the modules that can be imported and called.

References

N/A

Public - CC BY-SA 4.0 License

5.2. Unbounded JSON Nesting Leads to Denial of Service (DoS)

Severity	MEDIUM
Affected Resources	power-grid-model/power_grid_model_c/power_grid_model/include/power_grid_model/auxiliary/serialization/deserializer.hpp:121 power-grid-model/power_grid_model_c/power_grid_model/include/power_grid_model/auxiliary/serialization/deserializer.hpp:142
Status	Closed

August 20, 2026 Update

The vulnerability was reported via the GitHub Security section of the repository in the linked advisory and assigned CVE-2026-76912.
The vulnerability was fixed by implementing a `check_depth()` function to limit JSON and `msgpack` nesting to 10 elements. The fixes are implemented in PR #1467 and released in version 1.13.111.

Description

Power Grid Model's Python library allows users to parse JSON files containing models that will be used during calculations through the `json_deserialize()` method. Under the hood, `json_deserialize()` uses nlohmann's JSON parser to do the deserialization, passing the `JsonSAXVisitor` struct to it as visitor. When `nlohmann::json::sax_parse()` encounters a `[` or `{` token, it will call respectively `JsonSAXVisitor::start_array` and `JsonSAXVisitor::start_object`:

```
bool start_object(size_t /* elements */) {
    data_buffers.emplace();
    return true;
}
...
bool start_array(size_t /* elements */) {
    data_buffers.emplace();
    return true;
}
```

As it can be seen from the code, the buffer push lacks any depth check, meaning a deeply nested JSON payload could push however many entries inside the resulting object, effectively exhausting memory and getting the host process finally killed by the OOM-killer.

Impact

An attacker able to provide arbitrary data to the `json_deserialize()` method could achieve resource exhaustion and eventually Denial of Service (DoS) in the host process when the OOM-killer triggers.

Attack Complexity

The attacker needs to provide arbitrary JSON data to the vulnerable method and not to have limitations on the size of the serialized payload. It is important to notice that the

Public - CC BY-SA 4.0 License

exploitability of this vulnerability depends on the memory capabilities of the host process, knowing the payload has an amplification factor of ~4000x. During the tests, a payload of 2MB was used to crash the process on an Ubuntu Server machine.

Related Issues

N/A

Proof of Concept

1. Download the attached file `json_dos_poc.py`
2. From the directory with the file downloaded at step 1, run the PoC using `python3 json_dos_poc.py`
3. Notice the process being killed by the OOM-killer

Suggested Remediations

Add a depth counter to `JsonSAXVisitor` and check it before each `emplace()` in both `start_object()` and `start_array()`, throwing an exception if `data_buffers.size()` is higher than the maximum nesting depth accepted.

References

- https://cwe.mitre.org/data/definitions/770.html

Public - CC BY-SA 4.0 License

5.3. Out-Of-Bounds Memory Access in Span Calculation Leads to Denial of Service (DoS)

Severity	LOW
Affected Resources	power-grid-model /power_grid_model_c/power_grid_model/include/power_grid_model/auxiliary/dataset.hpp:645 power-grid-model /power_grid_model_c/power_grid_model/include/power_grid_model/auxiliary/dataset.hpp:597
Status	Closed

August 20, 2026 Update

The vulnerability was reported via the GitHub Security section of the repository in the linked advisory and assigned CVE-2026-76911.
The vulnerability was fixed by hardening the checks on the `indptr` array. The fixes are implemented in PR #1470 and released in version 1.13.115.

Description

The `PGM_dataset_const_add_buffer()` function allows the addition of a component buffer to a dataset. It also accepts non-uniform components; if a non-uniform component is added, the API will accept an `PGM_Idx indptr[]` array. Said `indptr` is an array of indexes that determines which slice of a total elements array belongs to which scenario:

```
indptr = {0, 2, 5, 7}
          ^  ^  ^  ^
          |  |  |  └─ scenario 2 ends at index 7
          |  |  └──── scenario 1 ends at index 5
          |  └─────── scenario 0 ends at index 2
          └────────── always 0 (start of buffer)
```

It is important, as stated in the API's documentation, that `indptr[0] = 0` and `indptr[batch_size] = total_elements`. The documentation also states that `indptr` elements should be non-decreasing. Although the first two constraints are checked by the `check_non_uniform_integrity()` C function, there are no checks to ensure:

1. that the monotonicity of the `indptr` array is enforced, making it possible to have, for example, `indptr = {0, 5, 2, 7}`
2. that no element goes Out-Of-Bounds for the total elements array, making it possible to have, for example, `indptr = {0, 100, 5, 7}`

Both cases will result in Out-Of-Bounds access when calculations are performed on the dataset.

It is important to notice, however, that this bug is not reachable when using the Python library, since the `indptr` vector is checked using the `check_indptr_consistency()` function that enforces monotonicity using the `diff()` method from the NumPy library.

Public - CC BY-SA 4.0 License

During the audit, no useful memory-read primitives were found by exploiting this issue, making Denial-of-Service the main impact of it.

Impact

An attacker able to pass a maliciously crafted `indptr` array to the `PGM_dataset_const_add_buffer()` function, could cause Out-of-Bounds memory access and crash the host process, obtaining Denial-of-Service.

Attack Complexity

The attacker needs to control the value of the `indptr` array passed to the `PGM_dataset_const_add_buffer()` function. It is hard to establish how common it would be for an attacker to have that level of control, thereby the severity of this vulnerability has been set to LOW.

Related Issues

N/A

Proof of Concept

1. Download the attached file `oob_poc.c` inside the `power-grid-model` repo root
2. Build the PGM library with ASAN using the following command:

   ```
   ./build.sh -p clang-debug
   ```

3. Build the PoC with ASAN using the following command:

   ```
   clang -fsanitize=address -g ./oob_poc.c -o oob_poc \
         -Ipower_grid_model_c/power_grid_model_c/include \
         -Lcpp_build/clang-debug/bin \
         -Wl,-rpath,'$ORIGIN/cpp_build/clang-debug/bin' \
         -lpower_grid_model_c
   ```

4. Run the PoC and observe the resulting ASAN crash report: `./oob_poc`

Suggested Remediations

Inside `check_non_uniform_integrity()`, perform a check on each element of the `indptr` array, ensuring that no element is larger than the total size and smaller than its predecessor. For example, a call to `std::is_sorted()` inside `dataset_const_add_buffer()` and/or `dataset_mutable_add_buffer()` could be enough to fulfill the requirement.

References

- https://cwe.mitre.org/data/definitions/125.html

Public - CC BY-SA 4.0 License

6. Hardenings

This section highlights recommendations to improve the security posture of the project.

6.1. pgm-build-dependencies Fetched via Mutable Latest Release

The build step "Install pgm-build-dependencies" from the Github workflows inside the `power-grid-model` repository fetches `https://github.com/PowerGridModel/pgm-build-dependencies/releases/latest/download/pgm_build_dependencies-0.1.0-py3-none-any.whl`. This is however a mutable reference to the latest release with no hash pinning. Therefore, if an attacker manages to compromise said release, they could be able to serve arbitrary code that runs on every `cibuildwheel` and `pip install`.

It is therefore advised to pin `pgm-build-dependencies` by SHA-256 hashing in `pyproject.toml` and in CI steps.

Public - CC BY-SA 4.0 License

7. Attachments

The following is the list of attachments and the related SHA256 integrity hashes.

7.1. *finding1/rce_poc.py*

Link: https://github.com/ShielderSec/poc/blob/main/power-grid-model/finding1/rce_poc.py

SHA256: `e27b816149e77fed64f40a85c80ad2c3087e22dbda4036166b64fc3e29952dbd`

7.2. *finding1/payload.yaml*

Link: https://github.com/ShielderSec/poc/blob/main/power-grid-model/finding1/payload.yaml

SHA256: `25d8c4e39a41de6efbe49b94443d91e3be9db49e5be666cb306a9a60805f4fa7`

7.3. *finding2/json_dos_poc.py*

Link: https://github.com/ShielderSec/poc/blob/main/power-grid-model/finding2/json_dos_poc.py

SHA256: `a89d6fc6149e93dcad1aaba8ef6c23b30d7f39d069989d20c0ce09153f38278c`

7.4. *finding3/oob_poc.c*

Link: https://github.com/ShielderSec/poc/blob/main/power-grid-model/finding3/oob_poc.c

SHA256: `4b7c1257b627bbd62f3ce7f55d7bf3dc40d17689d7957ca04fe73bbaf36c3e0d`

Public - CC BY-SA 4.0 License